INSTITUTO FEDERAL
Bahia

Disciplina: Lab. WEB I

Aluno: Gabriel Conrado da Silva

Docente: Eduardo Teles

SISTEMA GERADOR DE RELATÓRIO MENSAL DE RECEITAS BRUTAS PARA MEI

Camaçari

2025.2

SUMÁRIO

1. APRESENTAÇÃO…….3

2. TECNOLOGIAS UTILIZADAS…….3

3. PLATAFORMA DE IMPLEMENTAÇÃO……4

4. SISTEMA……..5

4.1 CAMADAS E PADRÃO DO PROJETO…….6

4.1.1 Modelo……..7

4.1.2 Visão…….10

4.1.3 Controlador…….11

4.2 TELAS……13

1. APRESENTAÇÃO

Este projeto apresenta o Sistema Gerador de Relatório Mensal de Receitas Brutas para MEI (Microempreendedor Individual), uma aplicação web desenvolvida em Java com Jakarta EE para auxiliar microempreendedores no controle financeiro de suas atividades e no cumprimento das obrigações fiscais previstas em lei.

O sistema permite o registro categorizado de vendas com ou sem emissão de Nota Fiscal, o acompanhamento histórico de receitas com filtros por período, categoria e status de NF, além da geração automatizada de relatórios mensais em formato PDF — documento utilizado pelo MEI na declaração anual obrigatória.

A relevância da temática se justifica pela dificuldade enfrentada pelos MEIs no controle do faturamento bruto anual, que não pode ultrapassar R$ 81.000,00 (limite legal vigente em 2026). O sistema propõe uma solução simples, funcional e de baixo custo para esse público.

2. TECNOLOGIAS UTILIZADAS

O sistema foi implementado utilizando a linguagem Java com Jakarta EE 10, servidor Apache Tomcat 10.1, banco de dados relacional MySQL e IDE Eclipse. A seguir, as tecnologias detalhadas:

- **Linguagens de marcação:** HTML5 (páginas JSP estruturadas com marcação HTML, como login.jsp, dashboard.jsp, historico.jsp, entre outras).
- **Linguagens de estilo:** CSS3 (arquivo sidebar.css responsável pela estilização global das interfaces, incluindo tema visual e layout responsivo).
- **Linguagens de programação:** Java 17 (back-end, servlets, modelos, DAOs e utilitários). JavaScript (vanilla, para interações na interface e validações de formulário, centralizado em validations.js).
- **Frameworks e bibliotecas:**

- Jakarta Servlet API (jakarta.servlet) para implementação de servlets, controle de sessão e processamento HTTP.
- JSP (JavaServer Pages) como tecnologia de visão para renderização dinâmica de páginas.
- JSTL (JavaServer Pages Standard Tag Library) para lógica de apresentação e iteração em JSP.
- MySQL Connector/J como driver JDBC para comunicação com o banco MySQL.
- jBCrypt para geração e verificação de hashes BCrypt no armazenamento seguro de senhas.
- iText para geração de relatórios mensais em formato PDF.

- **Banco de dados utilizado:** MySQL 8.0 (banco relacional, com suporte a transações ACID e chaves estrangeiras).
- **Plataformas:** Eclipse EE, Apache Maven, Apache Tomcat 10.1.

3. PLATAFORMA DE IMPLEMENTAÇÃO

O sistema foi desenvolvido como aplicação web Java, utilizando um ambiente de desenvolvimento integrado e um servidor de aplicação compatível com Servlets e JSP.

- **IDE utilizada:** Eclipse IDE, conforme evidenciado pelos metadados de projeto (.project, .classpath) e estrutura padrão Maven Web (src/main/java, src/main/webapp).
- **Servidor:** Apache Tomcat 10.1, configurado no diretório Servers e compatível com Jakarta Servlet API 6.0 e JSP 3.1.
- **Banco de dados:** MySQL 8.0, configurado por meio da classe utilitária Conexao.java com parâmetros de conexão JDBC (URL, usuário e senha).
- **Sistema operacional ou ambiente de execução:** Ambiente Java padrão (JVM) em servidor de aplicação Tomcat, em contexto típico de desenvolvimento local (localhost:8080/mei). O banco de dados foi executado localmente via XAMPP/Workbench 8.0 CE durante o desenvolvimento.

- **Outras ferramentas relevantes:**
    - Git (versionamento) com repositório remoto no GitHub.
    - Apache Maven para gerenciamento de dependências, com as bibliotecas jBCrypt, iText e MySQL Connector/J declaradas no pom.xml.

4. SISTEMA

O banco de dados do sistema possui seis tabelas inter-relacionadas para suportar o ciclo completo de vendas de um MEI. O DER abaixo ilustra a estrutura:

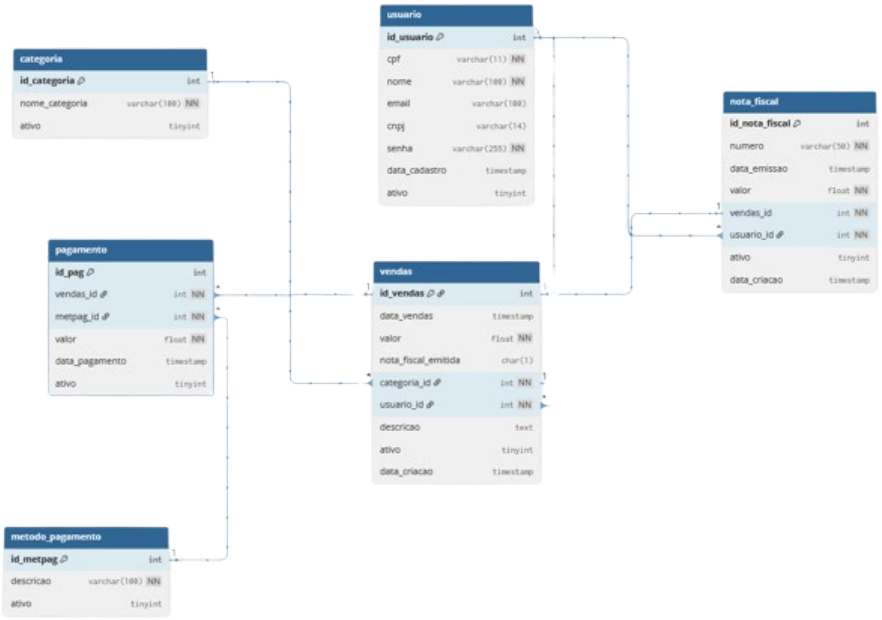

Figura 1 – DER do Banco de Dados MEI — MySQL/MariaDB

O banco de dados do sistema foi modelado em estilo relacional, contemplando as principais entidades: usuario, categoria, vendas, nota_fiscal, pagamento e metodo_pagamento. As tabelas são conectadas por chaves primárias numéricas e chaves estrangeiras que garantem integridade referencial entre usuários, categorias, vendas, notas fiscais e pagamentos.

As entidades principais são:

- **Tabela USUARIO:** Armazena os dados dos Microempreendedores Individuais cadastrados no sistema. Campos: id_usuario (PK), cpf, nome, email, cnpj, senha,

data_cadastro, ativo. Função: representar o MEI que acessa o sistema, utilizando CPF como identificador de login e senha armazenada em hash BCrypt.

- **Tabela CATEGORIA:** Armazena as categorias de classificação das vendas. Campos: id_categoria (PK), nome_categoria, ativo. Função: permitir que vendas sejam organizadas por tipo de atividade, viabilizando relatórios e análises por segmento.
- **Tabela VENDAS:** Registra cada venda realizada pelo MEI. Campos: id_vendas (PK), data_vendas, valor, nota_fiscal_emitida, categoria_id (FK), usuario_id (FK), descricao, ativo, data_criacao. Função: constituir o registro central do sistema, armazenando o histórico financeiro do MEI para fins de controle e geração de relatórios.
- **Tabela NOTA_FISCAL:** Armazena os dados das notas fiscais emitidas. Campos: id_nota_fiscal (PK), numero, data_emissao, valor, vendas_id (FK), usuario_id (FK), ativo, data_criacao. Função: manter o registro fiscal das vendas que exigiram emissão de nota, com relacionamento 1:1 com a tabela vendas.
- **Tabela PAGAMENTO:** Registra como cada venda foi paga. Campos: id_pag (PK), vendas_id (FK), metpag_id (FK), valor, data_pagamento, ativo. Função: permitir múltiplos pagamentos por venda, contemplando pagamentos mistos ou parcelados com diferentes métodos.
- **Tabela METODO_PAGAMENTO:** Armazena os métodos de pagamento disponíveis no sistema. Campos: id_metpag (PK), descricao, ativo. Função: permitir que pagamentos sejam associados a uma forma de recebimento específica, como Dinheiro, PIX, Cartão de Crédito, entre outros (função essa ultima ainda em desenvolvimento).

Relações e cardinalidades:

- USUARIO (1)–(N) VENDAS: um usuário pode ter muitas vendas, e cada venda pertence a exatamente um usuário via usuario_id.
- USUARIO (1)–(N) NOTA_FISCAL: um usuário pode ter muitas notas fiscais, e cada nota fiscal pertence a exatamente um usuário via usuario_id.
- CATEGORIA (1)–(N) VENDAS: uma categoria pode classificar muitas vendas, e cada venda referencia exatamente uma categoria via categoria_id.

- VENDAS (1)–(1) NOTA_FISCAL: uma venda pode ter no máximo uma nota fiscal, e cada nota fiscal está vinculada a exatamente uma venda via vendas_id (UNIQUE).
- VENDAS (1)–(N) PAGAMENTO: uma venda pode ter muitos pagamentos, e cada pagamento está vinculado a exatamente uma venda via vendas_id.
- METODO_PAGAMENTO (1)–(N) PAGAMENTO: um método de pagamento pode estar em muitos pagamentos, e cada pagamento referencia exatamente um método via metpag_id.

Os dados são manipulados pela camada de persistência (DAOs), que executa operações CRUD sobre essas tabelas e devolve objetos de modelo às camadas de controle e visão. A integridade do modelo é reforçada por restrições de chaves estrangeiras com ON DELETE CASCADE nos relacionamentos principais, garantindo que ao remover um usuário seus registros dependentes sejam também removidos, mantendo a consistência do histórico financeiro.

4.1 Camadas e Padrão do Projeto

O sistema utiliza o padrão MVC (Model-View-Controller) com a seguinte organização de pacotes:

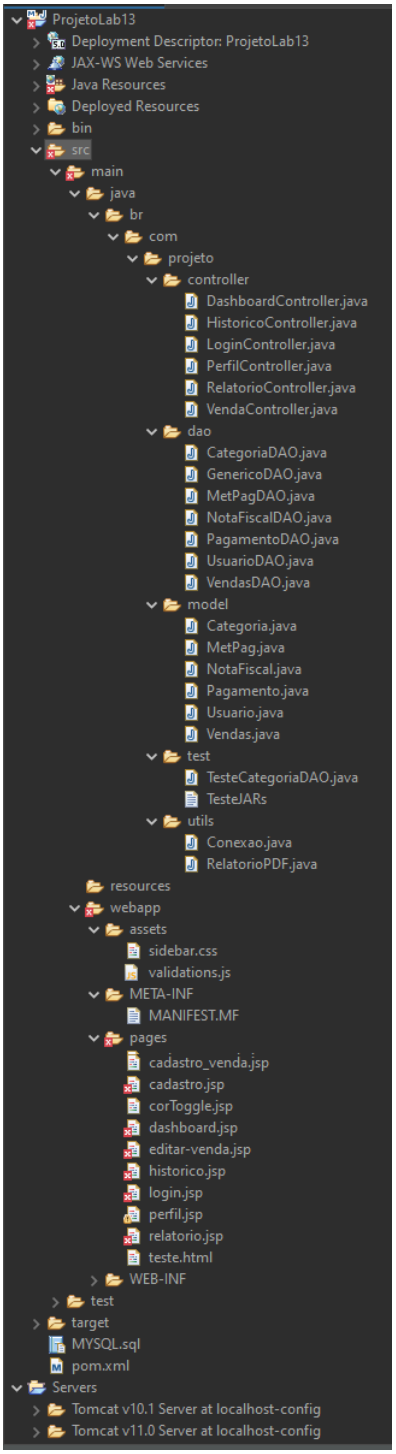

Figura 2 – Estrutura completa do projeto na IDE Eclipse

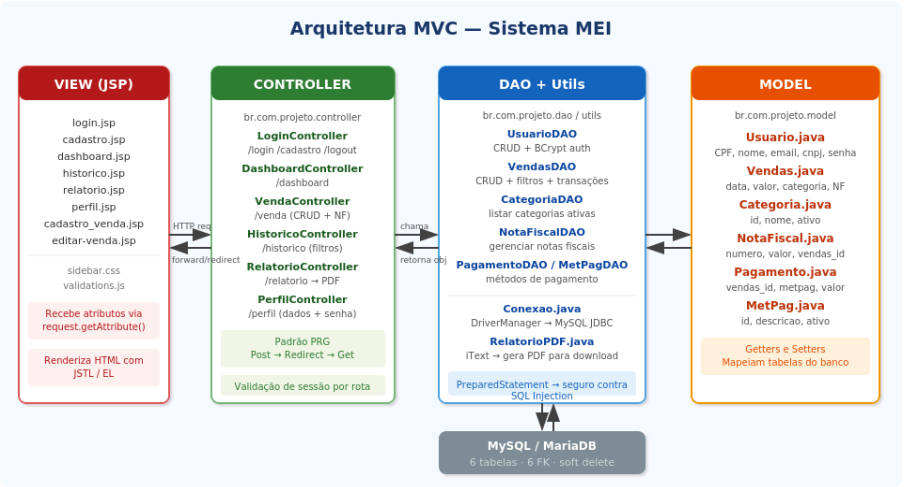

Figura 3 – Arquitetura MVC — pacotes, classes, rotas e tecnologias

Camada de Modelo/Persistência (Model)

Composta por classes de modelo em br.com.projeto.model (Categoria, MetPag, NotaFiscal, Pagamento, Usuario, Vendas) e classes de acesso a dados em br.com.projeto.dao (CategoriaDAO, GenericoDAO, MetPagDAO, NotaFiscalDAO, PagamentoDAO, UsuarioDAO, VendasDAO). Responsável por representar as entidades de domínio e encapsular a lógica de acesso ao banco de dados via JDBC.

Camada de Apresentação (View)

Implementada com JSP, HTML, CSS e JavaScript, localizada em src/main/webapp. Os arquivos de estilo e scripts estão em assets (sidebar.css, validations.js), e as páginas dinâmicas em pages (cadastro.jsp, corToggle.jsp, dashboard.jsp, editar-venda.jsp, historico.jsp, login.jsp, perfil.jsp, relatorio.jsp). Responsável por exibir dados ao usuário, receber entradas via formulários e renderizar interfaces com apoio de JSTL.

Camada de Controle (Controller)

Constituída por servlets Java, alocados em br.com.projeto.controller (DashboardController, HistoricoController, LoginController, PerfilController, RelatorioController, VendaController). Responsável por receber requisições HTTP, validar sessão, interpretar ações por parâmetro, coordenar chamadas aos DAOs e encaminhar respostas para as views.

Utilitários

Localizados em br.com.projeto.utils, o pacote contém Conexao.java, responsável pelo gerenciamento da conexão JDBC com o MySQL, e RelatorioPDF.java, responsável pela geração dos relatórios mensais em formato PDF via iText. O pacote test contém TesteCategoriaDAO.java e TesteJARs.java, utilizados para validação das dependências e operações de banco durante o desenvolvimento.

4.1.1 Modelo

Classes Model

As classes Model representam entidades do domínio e são utilizadas para transportar dados entre a camada de persistência e a camada de controle. Em geral, cada tabela do banco possui uma classe POJO (Plain Old Java Object — classe Java simples, criada apenas para representar dados, sem depender de frameworks ou regras complexas) correspondente, com atributos privados, construtores e métodos de acesso (getters/setters).

- **Classe Usuario.java:** Objetivo: representar um Microempreendedor Individual cadastrado no sistema, incluindo identificação, dados pessoais e credenciais de acesso. Dados representados: idUsuario, cpf, nome, email, cnpj, senha, dataCadastro, ativo. Comunicação: utilizada por UsuarioDAO para leitura e escrita no banco e pelo LoginController para autenticação via BCrypt, além de ser armazenada na sessão HTTP após login bem-sucedido e referenciada por VendaController, HistoricoController, RelatorioController e PerfilController.
- **Classe Categoria.java:** Objetivo: representar uma categoria de classificação de vendas. Dados representados: idCategoria, nomeCategoria, ativo. Comunicação: manipulada por CategoriaDAO e utilizada por VendaController e HistoricoController para popular os formulários de cadastro e filtros de histórico.
- **Classe Vendas.java:** Objetivo: representar uma venda realizada pelo MEI, sendo a entidade central do sistema. Dados representados: idVendas, dataVendas, valor, notaFiscalEmitida, categoriaId, usuarioId, descricao, ativo, dataCriacao, além de referências aos objetos Categoria e NotaFiscal. Comunicação: utilizada por VendasDAO para todas as operações CRUD, referenciada por DashboardController para totalizadores, por HistoricoController para listagem com filtros e por RelatorioController para geração do PDF mensal.

- **Classe NotaFiscal.java:** Objetivo: representar uma nota fiscal emitida e vinculada a uma venda. Dados representados: idNotaFiscal, numero, dataEmissao, valor, vendasId, usuarioId, ativo, dataCriacao. Comunicação: utilizada por NotaFiscalDAO e carregada junto ao objeto Vendas quando a venda possui nota fiscal emitida, sendo referenciada por VendaController no momento do cadastro e por RelatorioController na geração do relatório fiscal.
- **Classe Pagamento.java:** Objetivo: representar um registro de pagamento associado a uma venda, permitindo múltiplos pagamentos por venda. Dados representados: idPag, vendasId, metpagId, valor, dataPagamento, ativo. Comunicação: utilizada por PagamentoDAO e referenciada por VendaController no registro de formas de pagamento de cada venda.
- **Classe MetPag.java:** Objetivo: representar um método de pagamento disponível no sistema. Dados representados: idMetpag, descricao, ativo. Comunicação: utilizada por MetPagDAO e carregada por VendaController para popular os formulários de cadastro de venda com as opções de pagamento disponíveis.

Classes DAO

As classes DAO (Data Access Object) encapsulam toda a lógica de acesso ao banco de dados, executando operações SQL via JDBC e devolvendo objetos de modelo às camadas superiores. Cada DAO recebe um objeto Connection obtido a partir de Conexao.java e utiliza PreparedStatement em todas as operações para prevenir SQL injection.

- **UsuarioDAO.java:** Responsável pelas operações de banco relacionadas à entidade Usuario. Implementa inserir(), buscarPorCpf(), buscarPorId(), editar() e métodos de verificação de unicidade de CPF e email. É o DAO central para o fluxo de autenticação, sendo acionado pelo LoginController para localizar o usuário e validar a senha com BCrypt.
- **CategoriaDAO.java:** Responsável por listar, inserir e atualizar registros de categoria. Utilizado por VendaController e HistoricoController para carregar as opções disponíveis nos formulários e filtros da interface.

- **VendasDAO.java:** O DAO mais complexo do sistema. Implementa inserir() com transação atômica (commit/rollback) que persiste venda e nota fiscal em uma única operação, editar(), excluir() via soft delete, listarPorUsuario() e listarPorAnoEMesComFiltroNF() com construção dinâmica de query SQL baseada nos filtros informados pelo usuário.
- **NotaFiscalDAO.java:** Responsável por inserir, atualizar e buscar notas fiscais vinculadas a vendas. Opera em conjunto com VendasDAO dentro da mesma transação para garantir consistência entre os registros de venda e nota fiscal.
- **PagamentoDAO.java:** Responsável por registrar e listar pagamentos associados a vendas. Suporta múltiplos pagamentos por venda, permitindo pagamentos mistos ou parcelados.
- **MetPagDAO.java:** Responsável por listar os métodos de pagamento ativos disponíveis no sistema, utilizado para popular os formulários de cadastro de venda.
- **GenericoDAO.java:** Classe base com funcionalidades comuns compartilhadas entre os DAOs, como utilitários de mapeamento de ResultSet e tratamento de exceções, reduzindo duplicação de código na camada de persistência.

Além disso, a classe **Conexao.java** (em utils) abstrai a criação de conexões JDBC com o MySQL, sendo utilizada por todos os DAOs para obter objetos Connection via DriverManager. O padrão try-with-resources é adotado em todas as operações para garantir o fechamento automático das conexões após cada uso.

4.1.2 Visão

A camada View é composta por páginas JSP e recursos web que implementam a interface com o usuário. Os arquivos de apoio estão em src/main/webapp/assets (sidebar.css para estilização global e validations.js para validações no lado do cliente), e as páginas dinâmicas estão em src/main/webapp/pages.

Principais telas e funções:

- **login.jsp (Tela de Login):** Função: realizar a autenticação inicial do usuário, capturando CPF e senha e encaminhando-as ao LoginController. Conexão com

controlador: formulário HTML com método POST direcionado para /login. Exibe mensagem de erro genérica em caso de credenciais inválidas, sem revelar qual campo está incorreto.

- **cadastro.jsp (Cadastro de Usuário):** Função: permitir o registro de novos usuários MEI no sistema, coletando CPF, nome, email, CNPJ e senha com confirmação. Conexão com controlador: formulário POST direcionado para /cadastro, processado pelo LoginController, que valida unicidade de CPF e email antes de persistir o novo usuário com hash BCrypt.
- **dashboard.jsp (Painel Principal):** Função: apresentar visão geral financeira do MEI, com cards de totais de vendas do mês e do ano, listagem das últimas vendas e formulário de cadastro de venda com a função registrar uma nova venda com seleção de categoria, valor, descrição e opção de emissão de nota fiscal. Conexão com controlador: formulário POST direcionado para /venda, processado por VendaController, que persiste a venda e, se informada, a nota fiscal vinculada em transação atômica. Conexão com controlador: recebe atributos preenchidos por DashboardController (totais, lista de vendas recentes e categorias) e disponibiliza links para os demais módulos do sistema.
- **editar-venda.jsp (Edição de Venda):** Função: permitir a alteração dos dados de uma venda já registrada, com os campos pré-preenchidos com os valores atuais. Conexão com controlador: recebe o objeto Vendas carregado por VendaController via parâmetro acao=editar e envia os dados atualizados via POST para persistência.
- **historico.jsp (Histórico de Vendas):** Função: listar o histórico completo de vendas do MEI com filtros por ano, mês, categoria e status de nota fiscal, exibindo estatísticas totalizadoras de valor geral, com NF e sem NF. Conexão com controlador: recebe lista filtrada e totais calculados por HistoricoController, que aciona VendasDAO com query dinâmica baseada nos parâmetros informados.
- **relatorio.jsp (Relatório Mensal):** Função: permitir a seleção de período (mês e ano) para geração e download do relatório mensal de receitas brutas em formato PDF. Conexão com controlador: envia parâmetros de período para RelatorioController, que aciona RelatorioPDF.java via iText e devolve o arquivo ao navegador como download.
- **perfil.jsp (Perfil do Usuário):** Função: permitir a edição dos dados pessoais do MEI (nome, email, CNPJ) e a alteração de senha com verificação da senha atual via

BCrypt. Conexão com controlador: recebe o objeto Usuario da sessão carregado por PerfilController e envia os dados atualizados via POST para persistência.

- **corToggle.jsp:** Arquivo de apoio responsável pelo controle de alternância de tema visual (claro/escuro) da interface, utilizado como componente reutilizável pelas demais páginas.
- **teste.html:** Arquivo estático de apoio utilizado durante o desenvolvimento para testes de layout, dependências e configuração do ambiente, sem função no fluxo da aplicação em produção.

4.1.3 Controlador

A camada Controller é constituída por Servlets Java localizados em br.com.projeto.controller. Todos os controllers verificam a sessão HTTP no início de cada requisição, redirecionando para /login caso o usuário não esteja autenticado, e implementam o padrão PRG (Post-Redirect-Get) para evitar reenvio de formulários.

- **LoginController.java:** Mapeamento: /login, /cadastro, /logout. Função: gerenciar todo o ciclo de autenticação do sistema. No fluxo de login, lê o CPF informado, remove a máscara, busca o usuário via UsuarioDAO e valida a senha com BCrypt.checkpw(). Em caso de sucesso, cria a sessão HTTP e redireciona para /dashboard; em caso de falha, retorna mensagem de erro genérica. No fluxo de cadastro, valida unicidade de CPF e email, gera o hash BCrypt da senha e persiste o novo usuário via UsuarioDAO. No fluxo de logout, invalida a sessão e redireciona para /login.
- **DashboardController.java:** Mapeamento: /dashboard. Função: carregar os dados do painel principal do MEI. Busca os totais de vendas do mês e do ano corrente, as últimas vendas registradas e a lista de categorias ativas para o formulário rápido de nova venda. Encaminha todos os atributos para dashboard.jsp via request.setAttribute().
- **VendaController.java:** Mapeamento: /venda. Função: gerenciar o CRUD completo de vendas. No método GET, roteia a requisição pelo parâmetro acao: sem acao exibe o formulário de nova venda, acao=editar carrega os dados da venda para edição e

acao=excluir realiza o soft delete (ativo=0). No método POST, processa o cadastro ou edição da venda, incluindo a nota fiscal vinculada quando informada, via transação atômica em VendasDAO.

- **HistoricoController.java:** Mapeamento: /historico. Função: carregar o histórico de vendas do MEI com suporte a filtros combinados de ano, mês, categoria e status de nota fiscal. Lê os parâmetros da requisição, aciona VendasDAO.listarPorAnoEMesComFiltroNF() com query dinâmica e calcula os totalizadores (total geral, total com NF, total sem NF) para exibição em historico.jsp.
- **RelatorioController.java:** Mapeamento: /relatorio. Função: gerar o relatório mensal de receitas brutas em formato PDF no padrão oficial da DASN-SIMEI. No método GET, exibe o formulário de seleção de período (relatorio.jsp). No método POST, valida os parâmetros de mês (1–12) e ano, abre conexão via try-with-resources, busca as vendas do período via VendasDAO.listarPorMesAno() e verifica se há registros antes de prosseguir. Em seguida, aciona RelatorioPDF.calcularTotais() para obter o array de totais por categoria e nota fiscal, e RelatorioPDF.gerarRelatorio() para montar o documento em memória como byte[]. Configura o response HTTP com Content-Type: application/pdf, Content-Disposition: attachment com o nome do arquivo no formato relatorio_mei_[mes]_[ano].pdf e Content-Length, e escreve os bytes diretamente no OutputStream da resposta, entregando o arquivo como download sem salvá-lo em disco no servidor.
- **PerfilController.java:** Mapeamento: /perfil. Função: gerenciar a edição dos dados pessoais do MEI e a alteração de senha. No fluxo de edição de dados, atualiza nome, email e CNPJ via UsuarioDAO. No fluxo de alteração de senha, verifica a senha atual com BCrypt.checkpw() antes de gerar e persistir o novo hash, garantindo que apenas o próprio usuário possa alterar suas credenciais.

Utilitários (utils)

O pacote br.com.projeto.utils contém classes de suporte transversais ao sistema, utilizadas pelas camadas de controle e persistência.

- **Conexao.java:** Função: abstrair a criação e fornecimento de conexões JDBC com o banco de dados MySQL. Implementa o método estático getConnection(), que retorna

um objeto Connection via DriverManager utilizando as configurações de URL, usuário e senha. É utilizada por todos os DAOs no início de cada operação, sempre combinada com o padrão try-with-resources para garantir o fechamento automático da conexão após o uso, evitando vazamento de recursos.

- **RelatorioPDF.java:** Função: encapsular toda a lógica de geração de relatórios mensais de receitas brutas em formato PDF, utilizando a biblioteca iText, conforme o modelo oficial exigido pela Receita Federal do Brasil para o Microempreendedor Individual. Recebe os dados de vendas do período selecionado e produz um documento estruturado em três páginas: a primeira contém o cabeçalho com CNPJ e identificação do empreendedor, seguido da tabela de receitas brutas agrupadas por categoria com numeração oficial em algarismos romanos (itens I a XIV), separando valores com dispensa de emissão de documento fiscal dos valores com documento fiscal emitido, com subtotais por categoria e total geral do período; a segunda apresenta a declaração de documentos anexos e campo para local, data e assinatura do empresário; e a terceira exibe o detalhamento individual de cada venda registrada no período. O documento final é entregue ao usuário como download direto pelo RelatorioController.

4.2 Telas

As telas serão listadas em ordem lógica de uso e descrição das ações disponíveis ao usuário.

1. Tela de Login — login.jsp

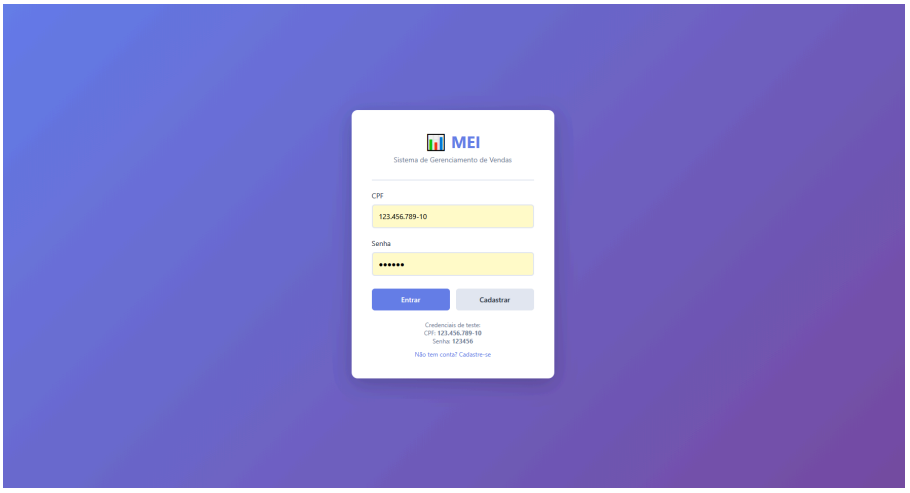

Figura 4 – Tela de Login

Esta tela tem como objetivo autenticar o usuário no sistema, funcionando como ponto de entrada da aplicação. Apresenta campos para CPF e Senha e botão de acesso. O usuário insere suas credenciais e aciona o botão "Entrar", o que dispara uma requisição POST ao LoginController; em caso de falha, uma mensagem de erro genérica é exibida na própria tela sem revelar qual campo está incorreto. A tela também disponibiliza link para a tela de cadastro de novo usuário.

2. Tela de Cadastro — cadastro.jsp

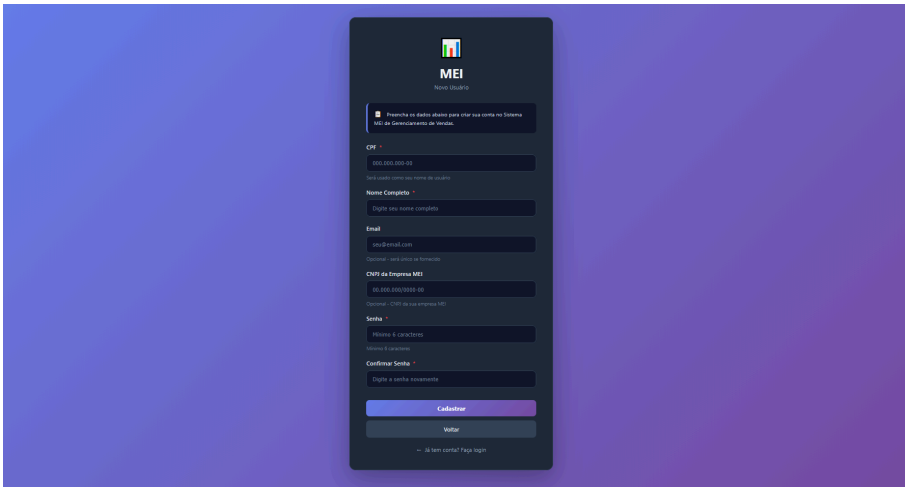

Figura 5 – Tela de Cadastro

Esta tela permite o registro de novos Microempreendedores Individuais no sistema. Apresenta campos para CPF, nome completo, email, CNPJ (opcional) e senha com confirmação. Validações de formato e unicidade são realizadas tanto no lado do cliente (validations.js)

quanto no servidor pelo LoginController. Ao concluir o cadastro com sucesso, o usuário é redirecionado para a tela de login com mensagem de confirmação.

3. Tela de Dashboard — dashboard.jsp

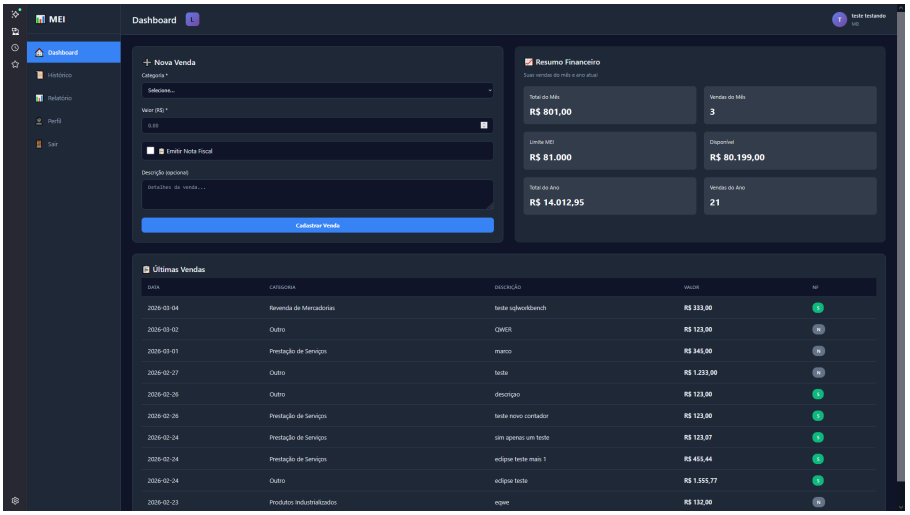

Figura 6 – Tela de Dashboard

A tela de dashboard fornece uma visão geral financeira do MEI após a autenticação. Apresenta cabeçalho com nome do usuário logado, menu lateral de navegação e cards com estatísticas de vendas do mês corrente e do ano, totais com e sem nota fiscal. Exibe também uma tabela com as últimas vendas registradas e um formulário rápido para cadastro de nova venda. As ações disponíveis incluem navegar para os demais módulos, visualizar indicadores financeiros e registrar uma venda diretamente pelo painel.

4. Tela de Histórico de Vendas — historico.jsp

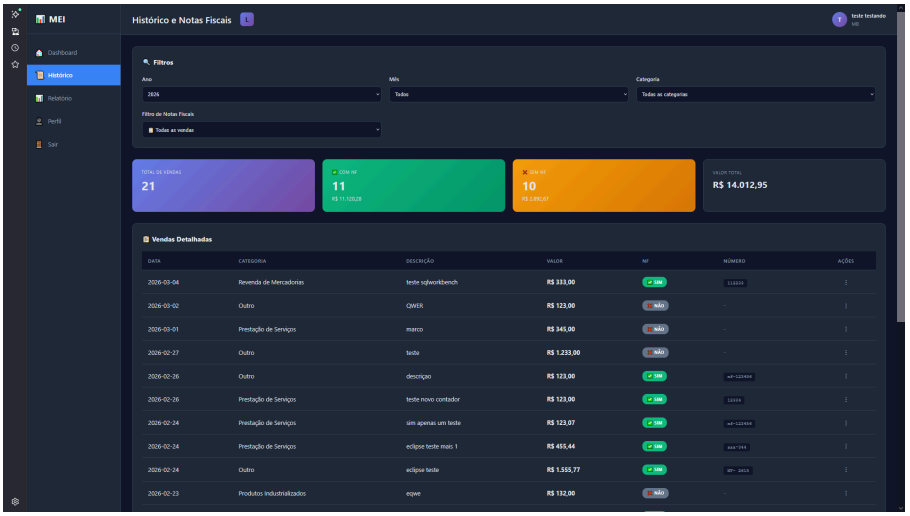

Figura 7 – Tela de Histórico de Vendas

Esta tela exibe o histórico completo de vendas do MEI com suporte a filtros combinados. O usuário pode filtrar por ano, mês, categoria e status de nota fiscal (com NF, sem NF ou todas). Os resultados são apresentados em tabela com colunas de data, descrição, categoria, valor e status de NF. Ao final da listagem são exibidos totalizadores de valor geral, total com nota fiscal e total sem nota fiscal. As ações disponíveis incluem aplicar filtros, editar venda e excluir venda.

5. Tela de Edição de Venda — editar-venda.jsp

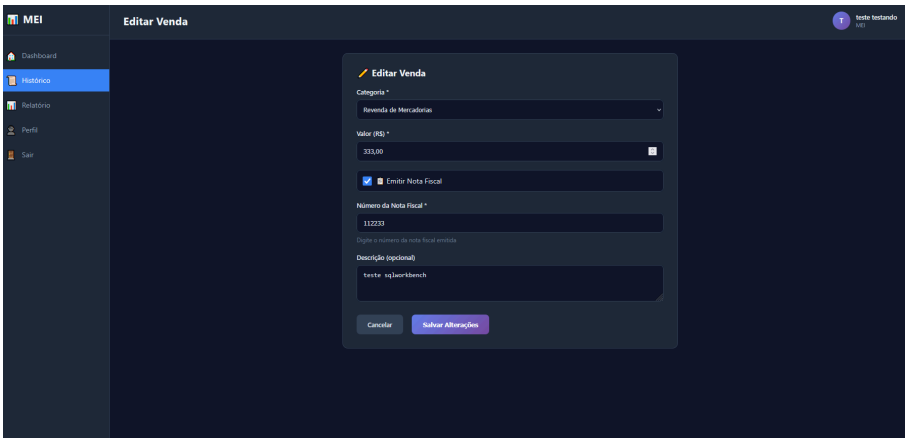

Figura 8 – Tela de Edição de Venda

Esta tela permite a alteração dos dados de uma venda já registrada. Os campos são pré-preenchidos com os valores atuais da venda selecionada, carregados pelo VendaController via parâmetro acao=editar. O usuário pode alterar categoria, valor, descrição, data e status de

nota fiscal. Ao confirmar, os dados atualizados são enviados via POST e persistidos via VendasDAO.

6. Tela de Relatório Mensal — relatorio.jsp

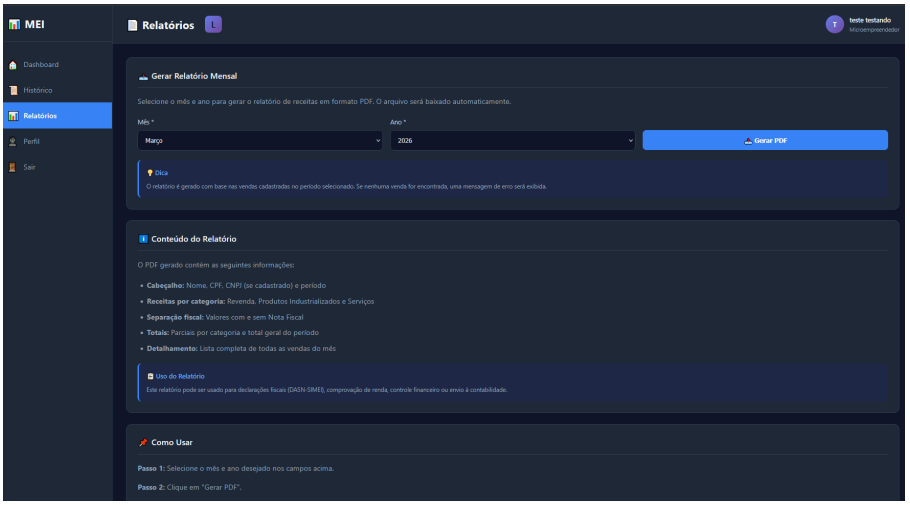

Figura 9 – Tela de Relatório Mensal

A tela de relatório permite ao MEI gerar e baixar o relatório mensal de receitas brutas em formato PDF, documento utilizado como base para a declaração anual obrigatória. O usuário seleciona o mês e o ano desejados e aciona o botão de geração. O sistema busca as vendas do período via VendasDAO, calcula os totais e aciona RelatorioPDF.java para montar e entregar o arquivo via download.

7. Tela de Perfil do Usuário — perfil.jsp

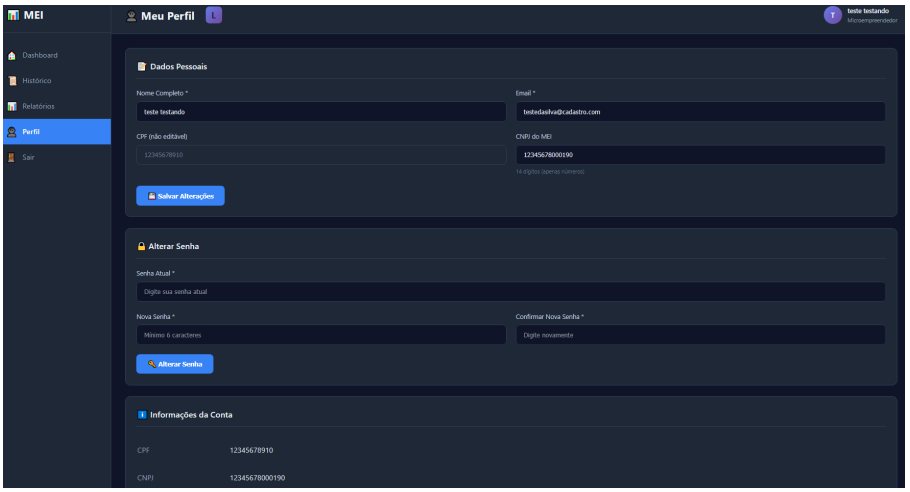

Figura 10 – Tela de Perfil do Usuário

A tela de perfil concentra as informações cadastrais do MEI logado e permite sua atualização. O usuário pode editar nome, email e CNPJ, e também realizar a alteração de senha informando a senha atual para verificação via BCrypt antes de definir a nova. As alterações são processadas pelo PerfilController e persistidas via UsuarioDAO, com redirecionamento e mensagem de confirmação ao concluir.

8. Exemplo de Relatório Gerado — RelatorioPDF.java

RELATÓRIO MENSAL DAS RECEITAS BRUTAS

CNPJ: 12.345.678/0001-90

Empreendedor individual: teste testando

Período de apuração: Fevereiro/2026

RECEITA BRUTA MENSAL – REVENDA DE MERCADORIAS (COMÉRCIO)

I – Revenda de mercadorias com dispensa de emissão de documento fiscal	R$ 9,00
II – Revenda de mercadorias com documento fiscal emitido	R$ 0,00
III – Total das receitas com revenda de mercadorias (I + II)	**R$ 9,00**

RECEITA BRUTA MENSAL – VENDA DE PRODUTOS INDUSTRIALIZADOS (INDÚSTRIA)

IV – Venda de produtos industrializados com dispensa de emissão de documento fiscal	R$ 687,66
V – Venda de produtos industrializados com documento fiscal emitido	R$ 1.341,00
VI – Total das receitas com venda de produtos industrializados (IV + V)	**R$ 2.028,66**

RECEITA BRUTA MENSAL – PRESTAÇÃO DE SERVIÇOS

VII – Receita com prestação de serviços com dispensa de emissão de documento fiscal	R$ 1.718,01
VIII – Receita com prestação de serviços com documento fiscal emitido	R$ 7.367,51
IX – Total das receitas com prestação de serviços (VII + VIII)	**R$ 9.085,52**

RECEITA BRUTA MENSAL – OUTRO

XI – Outras receitas com dispensa de emissão de documento fiscal	R$ 0,00
XII – Outras receitas com documento fiscal emitido	R$ 2.078,77
XIII – Total das outras receitas (XI + XII)	**R$ 2.078,77**

XIV - Total geral das receitas brutas no mês (III + VI + IX + XIII)	**R$ 13.201,95**

Figura 11 – Exemplo de Relatório Mensal de Receitas Brutas em PDF - parte 1

LOCAL E DATA: _____ ASSINATURA DO EMPRESÁRIO: _____

ENCONTRAM-SE ANEXADOS A ESTE RELATÓRIO:
- Os documentos fiscais comprobatórios das entradas de mercadorias e serviços tomados referentes ao período;
- As notas fiscais relativas às operações ou prestações realizadas eventualmente emitidas.

DETALHAMENTO DAS VENDAS

Data	Categoria	Descrição	NF	Número	Valor
09/02/2026	Prestação de Serviços	inventario	N	-	R$ 345,00
09/02/2026	Prestação de Serviços	teste	N	-	R$ 34,00
09/02/2026	Produtos Industrializados	limpeza	N	-	R$ 555,66
09/02/2026	Revenda de Mercadorias	pao	N	-	R$ 9,00
09/02/2026	Prestação de Serviços	aaa	N	-	R$ 5,00
09/02/2026	Prestação de Serviços	nao	N	-	R$ 111,01
09/02/2026	Outro	ssss	S	NF-54312	R$ 222,00
11/02/2026	Outro	aaaa	S	NF-543124	R$ 55,00
14/02/2026	Prestação de Serviços	TESTE	S	nf-123456	R$ 6.666,00
15/02/2026	Produtos Industrializados	teste	S	nf-123123	R$ 1.341,00
23/02/2026	Outro	teste	S	nf-1231233	R$ 123,00
23/02/2026	Produtos Industrializados	eqwe	N	-	R$ 132,00
24/02/2026	Outro	eclipse teste	S	NF- 2615	R$ 1.555,77
24/02/2026	Prestação de Serviços	eclipse teste mais 1	S	aaa-344	R$ 455,44
24/02/2026	Prestação de Serviços	sim apenas um teste	S	nf-123456	R$ 123,07
26/02/2026	Prestação de Serviços	teste novo contador	S	12334	R$ 123,00
26/02/2026	Outro	descriçao	S	nf-123456	R$ 123,00
27/02/2026	Prestação de Serviços	teste	N	-	R$ 1.223,00

Figura 11 – Exemplo de Relatório Mensal de Receitas Brutas em PDF - parte 2

O sistema gera automaticamente um relatório mensal de receitas brutas em formato PDF, seguindo o modelo oficial exigido pela Receita Federal do Brasil para o Microempreendedor Individual. O documento é produzido pela classe utilitária RelatorioPDF.java via biblioteca iText e entregue ao usuário como download pelo RelatorioController, sem ser armazenado em disco no servidor.

O relatório é estruturado em três páginas. A primeira página apresenta o cabeçalho com o título "RELATÓRIO MENSAL DAS RECEITAS BRUTAS", CNPJ do empreendedor, nome completo identificado como "Empreendedor individual" e o período de apuração no formato Mês/Ano. Em seguida, organiza as receitas do mês em quatro blocos por categoria — Revenda de Mercadorias (Comércio), Venda de Produtos Industrializados (Indústria), Prestação de Serviços e Outro — cada bloco contendo dois itens numerados em algarismos romanos: o primeiro referente a receitas com dispensa de emissão de documento fiscal (vendas sem NF) e o segundo referente a receitas com documento fiscal emitido (vendas com NF), seguidos de um subtotal por categoria. A estrutura segue a numeração oficial da DASN-SIMEI, com itens de I a XIV, incluindo os subtotais por categoria (III, VI, IX e XIII) e o total geral das receitas brutas do mês no item XIV, calculado pela soma dos quatro subtotais.

A segunda página contém a declaração de documentos anexos ao relatório — incluindo comprovantes de entradas e notas fiscais de operações realizadas no período — seguida de campos para local, data e assinatura do empresário, conferindo validade documental ao relatório para fins de auditoria fiscal.

A terceira página apresenta o detalhamento individual de cada venda registrada no período, em tabela com colunas de data, categoria, descrição, status de emissão de nota fiscal, número da nota fiscal quando aplicável e valor da venda, ordenadas cronologicamente. O detalhamento permite rastrear cada operação que compõe os totais apresentados na primeira página.

.